# Exercise 2 – Regression

## Introduction to Machine Learning

*Hint: Useful libraries*

### R

```r
# Consider the following libraries for this exercise sheet:

library(ggplot2)
library(mlr3verse)
library(mlr3learners)
library(mlr3viz)
library(quantreg)
```

### Python

```python
# Consider the following libraries for this exercise sheet:

# general
import numpy as np
import pandas as pd
import math

# plots
import matplotlib.pyplot as plt
import seaborn as sns

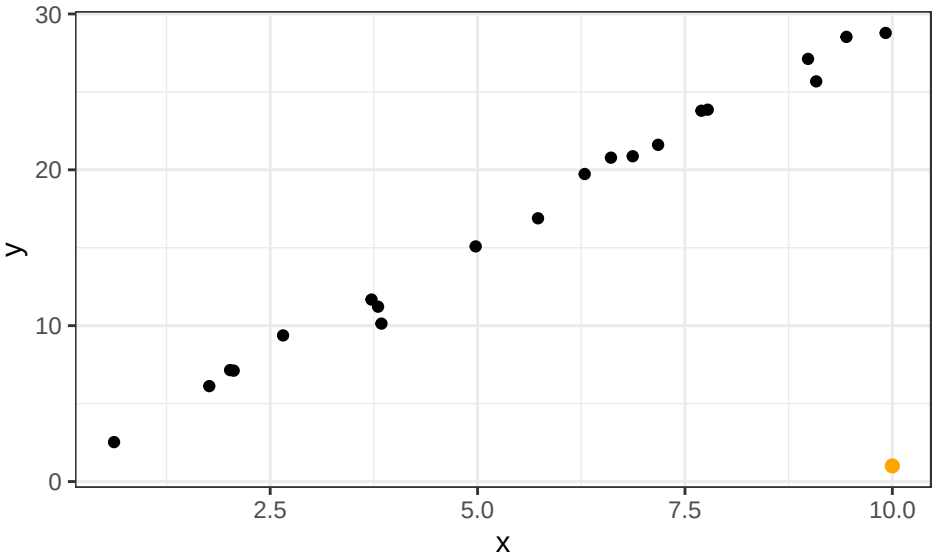
# sklearn
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeRegressor
from sklearn.linear_model import LinearRegression
import sklearn.metrics as metrics
from sklearn.metrics import mean_absolute_error
```

## Exercise 1: Loss functions for regression tasks

Learning goals

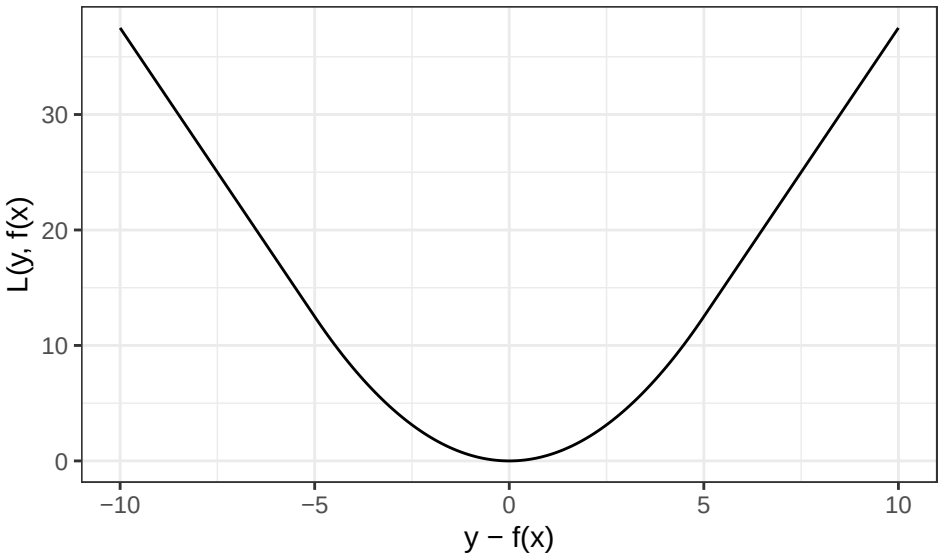
1. Assess how outliers affect models for different loss functions
2. Derive impact of a loss function from visual representation

In this exercise, we will examine loss functions for regression tasks somewhat more in depth.




a) Consider the above linear regression task. How will the model parameters be affected by adding the new outlier point (orange) if you use $L1$ loss and $L2$ loss, respectively, in the empirical risk? (You do not need to actually compute the parameter values.)

b) The second plot visualizes another loss function popular in regression tasks, the so-called *Huber loss* (depending on $\epsilon > 0$; here: $\epsilon = 5$). Describe how the Huber loss deals with residuals as compared to $L1$ and $L2$ loss. Can you guess its definition?




## Exercise 2: HRO in coding frameworks

Learning goals

1. Translate lecture concepts to code
2. Evaluate a model using a custom loss function

Throughout the lecture, we will frequently use the `R` package `mlr3`, resp. the `Python` package `sklearn`, and its descendants, providing an integrated ecosystem for all common machine learning tasks. Let's recap the HRO principle and see how it is reflected in either `mlr3` or `sklearn`. An overview of the most important objects and their usage, illustrated with numerous examples, can be found at the `mlr3` book and the `scikit` documentation.

a) How are the key concepts (i.e., hypothesis space, risk and optimization) you learned about in the lecture videos implemented?

b) Have a look at `mlr3::tsk("iris")` / `sklearn.datasets.load_iris`. What attributes does this object store?

c) Instantiate a regression tree learner (`lrn("regr.rpart")` / `DecisionTreeRegressor`). What are the different settings for this learner?

*Hint*

**R**

`mlr3::mlr_learners$keys()` shows all available learners.

**Python**

Use `get_params()` to see all available settings.

d) In Exercise 1, you encountered the *Huber loss*. Unlike $L1$ and $L2$ loss, it is not shipped as a ready-made loss/measure in either `mlr3` or `sklearn`. An implementation (following the definition from Exercise 1) is provided below. Use it to compute the empirical Huber risk (the mean Huber loss over all residuals) of the linear model from a). How does it compare to the MSE, and what does the difference tell you about how the two measures treat large residuals? In `sklearn`, you can wrap the loss into a custom scorer via `make_scorer`.

**R**

```r
# Given: the Huber loss (neither mlr3 nor sklearn ships it out of the box).
# It behaves like L2 loss for small residuals (|r| <= epsilon) and like L1 for
# large ones, combining smoothness with robustness to outliers.
huber_loss <- function(residual, epsilon = 1) {
  ifelse(
    abs(residual) <= epsilon,
    0.5 * residual^2,
    epsilon * abs(residual) - 0.5 * epsilon^2
  )
}
```

**Python**

```
# Given: the Huber loss (neither mlr3 nor sklearn ships it out of the box).
# It behaves like L2 loss for small residuals (|r| <= epsilon) and like L1 for
# large ones, combining smoothness with robustness to outliers.
def huber_loss(residual, epsilon=1.0):
    residual = np.asarray(residual, dtype=float)
    is_small = np.abs(residual) <= epsilon
    return np.where(
        is_small,
        0.5 * residual**2,
        epsilon * np.abs(residual) - 0.5 * epsilon**2,
    )
```

## Exercise 3: Polynomial regression

The whole exercise is only for lecture group A!

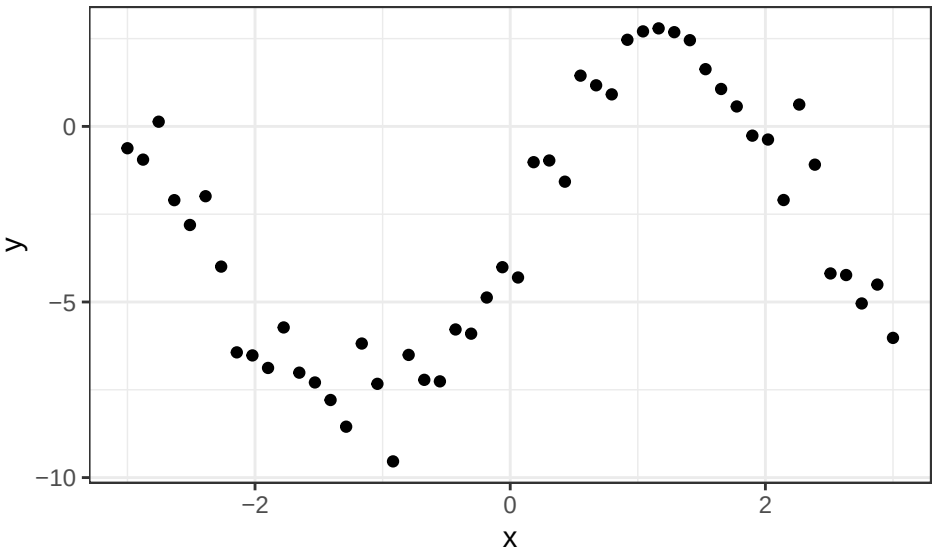
Learning goals

1. Express HRO components for polynomial regression
2. Derive gradient update for optimization
3. Analyze and discuss learner flexibility

Assume the following (noisy) data-generating process from which we have observed 50 realizations:

$$y = -3 + 5 \cdot \sin(0.4\pi x) + \epsilon$$

with $\epsilon \sim \mathcal{N}(0, 1)$.




a) We decide to model the data with a cubic polynomial (including intercept term). State the corresponding hypothesis space.

b) State the empirical risk w.r.t. $\theta$ for a member of the hypothesis space. Use $L2$ loss and be as explicit as possible.

c) We can minimize this risk using gradient descent. Derive the gradient of the empirical risk w.r.t $\theta$.

d) Using the result for the gradient, explain how to update the current parameter $\theta^{[t]}$ in a step of gradient descent.

e) You will not be able to fit the data perfectly with a cubic polynomial. Describe the advantages and disadvantages that a more flexible model class would have. Would you opt for a more flexible learner?

## Exercise 4: Fitting a linear model by hand

The whole exercise is only for lecture group B!

Learning goals

1. Compute the least-squares estimator on a small example by hand
2. Interpret linear model coefficients
3. Predict with a fitted linear model

In the lecture, you saw that a linear model with $L2$ loss has a closed-form solution – the *ordinary-least-squares (OLS)* estimator

$$\hat{\theta} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y},$$

where $\mathbf{X}$ is the design matrix (the feature columns padded with a leading column of ones for the intercept) and $\mathbf{y}$ the vector of observed targets. Consider the following small dataset with a single feature $x$ and target $y$:

| $i$ | $x^{(i)}$ | $y^{(i)}$ |
|---|---|---|
| 1 | $-3$ | 1 |
| 2 | $-1$ | 1 |
| 3 | 1 | 3 |
| 4 | 3 | 7 |

We model it with a simple linear regression $f(x) = \theta_0 + \theta_1 x$.

a) Set up the design matrix $\mathbf{X}$ and compute the OLS estimator $\hat{\theta} = (\hat{\theta}_0, \hat{\theta}_1)^\top$ by hand.

*Hint*

The feature is centered ($\sum_i x^{(i)} = 0$), which makes $\mathbf{X}^\top \mathbf{X}$ diagonal and hence trivial to invert.

b) Interpret the two fitted coefficients $\hat{\theta}_0$ and $\hat{\theta}_1$.

c) Use the fitted model to predict the target for a new observation with $x = 2$.

## Exercise 5: Predicting `abalone`

Bonus exercise – will not be discussed in the tutorial session

This optional exercise is meant for both lecture groups.

Learning goals

1. Implement a regression model
2. Interpret linear model coefficients
3. Analyze basic regression fit

We want to predict the age of an abalone using its longest shell measurement and its weight. The `abalone` data can be found here: https://archive.ics.uci.edu/ml/machine-learning-databases/abalone/abalone.data.

Prepare the data as follows:

### R

```r
# Download data
url <- "https://archive.ics.uci.edu/ml/machine-learning-databases/abalone/abalone.data"
abalone <- read.table(url, sep = ",", row.names = NULL)
colnames(abalone) <- c(
  "sex",
  "longest_shell",
  "diameter",
  "height",
  "whole_weight",
  "shucked_weight",
  "visceral_weight",
  "shell_weight",
  "rings"
)

# Reduce to relevant columns
abalone <- abalone[, c("longest_shell", "whole_weight", "rings")]
```

### Python

```python
# load data from url

url = "https://archive.ics.uci.edu/ml/machine-learning-databases/abalone/abalone.data"
```

```
abalone = pd.read_csv(
    url,
    sep=",",
    names=[
        "sex",
        "longest_shell",
        "diameter",
        "height",
        "whole_weight",
        "shucked_weight",
        "visceral_weight",
        "shell_weight",
        "rings",
    ],
)

abalone = abalone[["longest_shell", "whole_weight", "rings"]]
print(abalone.head)
```

```
<bound method NDFrame.head of       longest_shell  whole_weight  rings
0              0.455        0.5140     15
1              0.350        0.2255      7
2              0.530        0.6770      9
3              0.440        0.5160     10
4              0.330        0.2050      7
...              ...           ...    ...
4172           0.565        0.8870     11
4173           0.590        0.9660     10
4174           0.600        1.1760      9
4175           0.625        1.0945     10
4176           0.710        1.9485     12

[4177 rows x 3 columns]>
```

a) Plot `LongestShell` and `WholeWeight` on the $x$- and $y$-axis, respectively, and color points according to `Rings`.

b) Using `mlr3`/`sklearn`, fit a linear regression model to the data.

c) A key advantage of the linear model is that its parameters are directly interpretable. Read off the fitted coefficients and interpret the effect of `longest_shell` on the predicted number of rings.

d) Compare the fitted and observed targets visually.

*R Hint*

**R**

Use `$autoplot()` from `mlr3viz`.

e) Assess the model's training loss in terms of MAE.

*Hint*

**R**

Call `$score()`, which accepts different `mlr_measures`, on the prediction object.

**Python**

Call `from sklearn.metrics import mean_absolute_error`.